

Tách Từ (Word Tokenization)

Xử Lý Ngôn Ngữ Tự Nhiên
(Natural Language Processing - NLP)

Vũ Đình Khôi

Nội dung

- Biểu thức chính quy (Regular expressions)
- Quy tắc cấu tạo từ tiếng Anh
- Các hướng tiếp cận tách từ trong văn bản tiếng Anh
- Một số công cụ tách từ tiếng Anh
- Quy tắc cấu tạo từ tiếng Việt
- Các hướng tiếp cận tách từ trong văn bản tiếng Việt
- Một số công cụ tách từ tiếng Việt

- [1] Chapter 2. Dan Jurafsky and James H. Martin, *Speech and Language Processing*, 3rd ed. Pearson, 2026.
 - <https://web.stanford.edu/~jurafsky/slp3/>
- [2] Sowmya Vajjala, Bodhisattwa Majumder, Anuj Gupta, and Harshit Surana, *Practical Natural Language Processing*. O'Reilly, 2020.
- [3] Chapter 2. Jay Alammara and Maarten Grootendorst, *Hands-On Large Language Models*. O'Reilly, 2024.
- [4] Chapter 3. Suhas Pai, *Designing Large Language Model Applications*. O'Reilly, 2025.

Chỉ cần đọc các Chapter được liệt kê

I find that I don't understand things unless I try to program them.



Donald E. Knuth

Professor Emeritus at Stanford University

If you want to **master something**, teach it



-- **Richard Feynman**

ASCII

▪ American Standard Code for Information Interchange (ASCII)

- Thập niên 1960

▪ Biểu diễn các ký tự/chữ cái Latin (Latin characters)

- Mỗi ký tự 1 byte → tối đa 256 ký tự
- Nhưng bit giá trị cao nhất (high-order) luôn là 0 → Còn 7 bits → Có tất cả 128 ký tự
 - Thực tế chỉ dùng 95, còn lại là các “ký tự điều khiển” (control codes) của thiết bị teletypes

Ch	Hex	Dec	Ch	Hex	Dec	Ch	Hex	Dec	Ch	Hex	Dec	
<	3C	60	@	40	64	...	\	5C	92	'	60	96
=	3D	61	A	41	65	...	[	5D	93	a	61	97
>	3E	62	B	42	66	...	^	5E	94	b	62	98
?	3F	63	C	43	67	...	_	5F	95	c	63	99

h	e	l	l	o
68	65	6C	6C	6F

→ Không đủ cho các hệ thống chữ viết (writing systems) trên thế giới

- “Tôi thích tìm hiểu Xử lý ngôn ngữ tự nhiên.”
- Câu trên biểu diễn thế nào?

ASCII: thiết bị teletypes



■ Phương pháp biểu diễn văn bản viết bằng:

- bất cứ ký tự (character) nào (hơn 150,000 ký tự)
- trong bất kỳ hệ chữ viết (script) nào (hơn 168 hệ thống chữ viết)
- của các ngôn ngữ (language) trên thế giới
 - tiếng Trung (Chinese), Ả-rập (Arabic), Hindi, Khmer, tiếng Việt (Vietnamese)...
 - các hệ chữ đã tuyệt chủng như chữ hình nêm Sumer
 - các hệ chữ đã tuyệt chủng như chữ hình nêm Sumer
 - các hệ chữ được sáng tạo như tiếng Klingon
 - cộng thêm biểu tượng cảm xúc (emojis), ký hiệu tiền tệ...



Unicode: code points

- Mỗi ký tự trong bảng mã Unicode được gán một mã số duy nhất (**unique id**) được gọi là **code point**.
 - Gán cho 150,000 ký tự
 - Có khoảng 1.1 triệu code points khả dụng (từ 0 – 0x10FFFF)
 - Dạng hex với tiền tố (prefix) U+
 - Ví dụ ký tự a là U+0061 (0x0061)
 - 127 code points đầu tiên tương ứng với ASCII

U+0061	a	LATIN SMALL LETTER A
U+0062	b	LATIN SMALL LETTER B
U+0063	c	LATIN SMALL LETTER C
U+00F9	ù	LATIN SMALL LETTER U WITH GRAVE
U+00FA	ú	LATIN SMALL LETTER U WITH ACUTE
U+00FB	û	LATIN SMALL LETTER U WITH CIRCUMFLEX
U+00FC	ü	LATIN SMALL LETTER U WITH DIAERESIS
U+8FDB	进	
U+8FDC	远	
U+8FDD	违	
U+8FDE	连	
U+1F600	😊	GRINNING FACE
U+1F00E	八萬	MAHJONG TILE EIGHT OF CHARACTERS

Unicode: UTF-8 encoding

- Chúng ta không lưu trực tiếp giá trị code point vào tập tin
 - Lưu mã hóa (encodings) của các ký tự
- Unicode không quy định cụ thể số byte, mà có nhiều cách mã hóa khác nhau
 - UTF-8 (Unicode Transformation Format 8)
 - phổ biến, tương thích với ASCII, tiết kiệm dung lượng [lưu trữ]
 - 1 - 4 bytes/ký tự (các ký tự Latin cơ bản chiếm 1 byte) → variable-length encoding
 - Tiếng Việt thường 2-3 bytes
 - UTF-16: 2 hoặc 4 bytes/ký tự
 - Hầu hết các ký tự tiếng Việt chiếm 2 bytes
 - UTF-16: 4 bytes cố định
- Ví dụ, UTF-8
 - Chữ A trong ASCII tương ứng với A (1 byte) trong UTF-8

Unicode: UTF-8 encoding

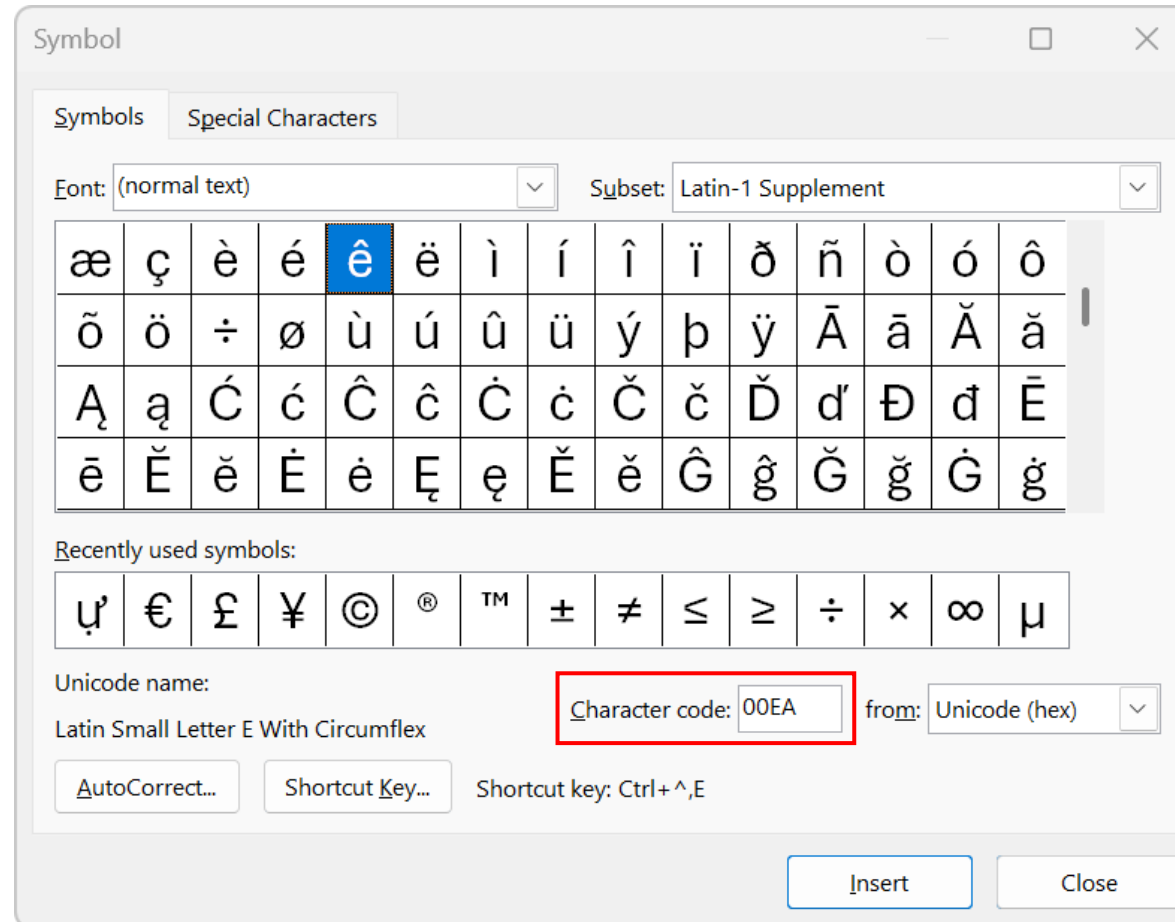
- Vì số bytes biểu diễn không cố định → Làm sao xác định giới hạn mỗi ký tự trong dãy các bytes liên tục?
 - Code points có giá trị trong khoảng nào → xác định số byte(s) dùng để mã hóa

Code Points		UTF-8 Encoding			
From - To	Bit Value	Byte 1	Byte 2	Byte 3	Byte 4
U+0000 - U+007F	0xxxxxxx	0xxxxxxx			
U+0080 - U+07FF	00000yyy yyxxxxxx	110yyyyy	10xxxxxx		
U+0800 - U+FFFF	zzzzyyyy yyxxxxxx	1110zzzz	10yyyyyy	10xxxxxx	
U+10000 - U+10FFFF	000uuuuu zzzzyyyy yyxxxxxx	11110uuu	10uuzzzz	10yyyyyy	10xxxxxx

- ê → code point U+00EA trong khoảng U+0080 - U+07FF → 2 bytes
- ự → code point U+1EE5 → 3 bytes

Unicode: UTF-8 encoding

- Bảng Symbol (phần mềm Microsoft Word)
 - ê → code point U+00EA

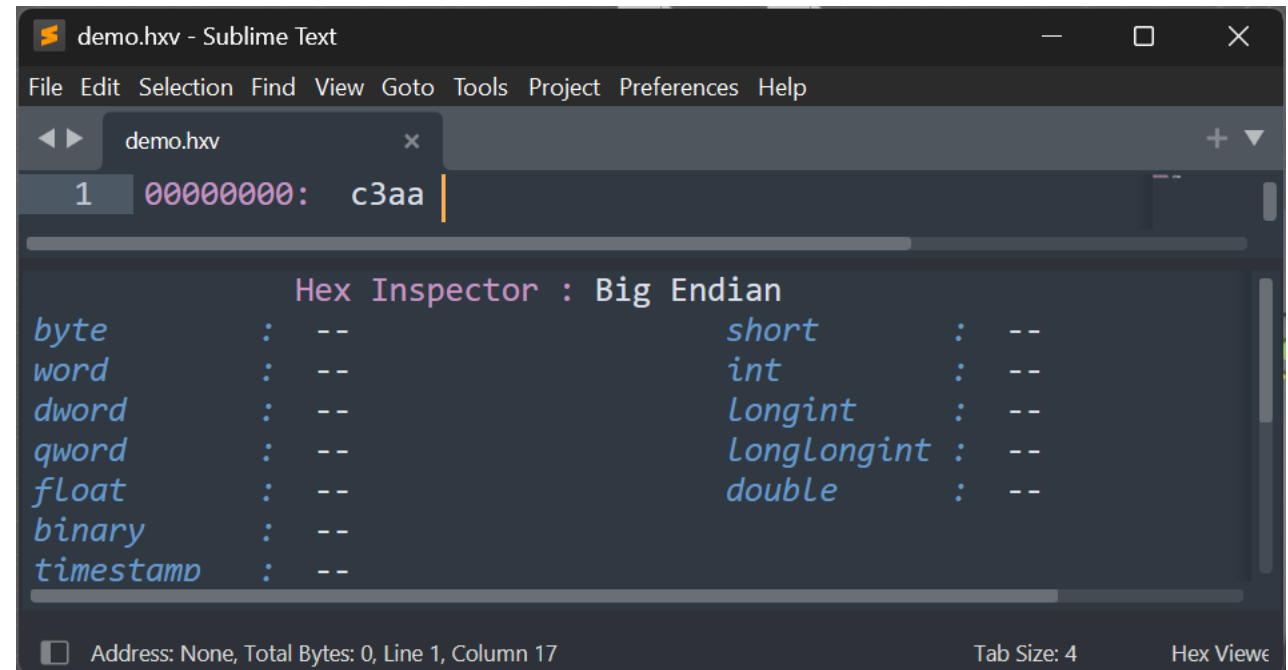
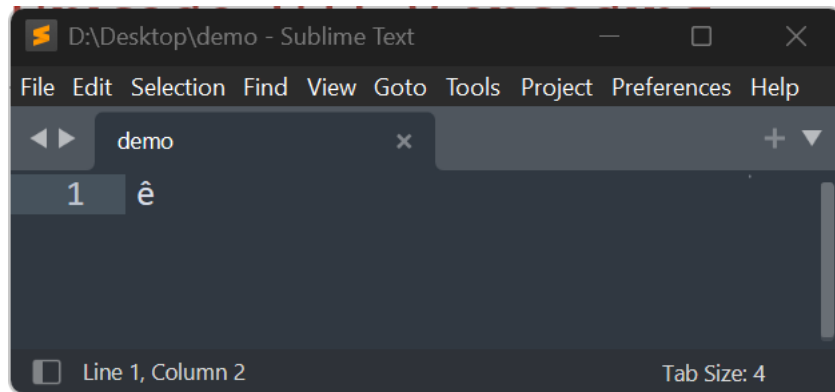


Unicode: UTF-8 encoding

- Chúng ta không lưu trực tiếp giá trị code point vào tập tin
 - Lưu mã hóa (encodings) của các ký tự
- Biết code point của ký tự ê là U+00EA làm sao biết UTF-8 encoding của nó?
 - U+00EA → biểu diễn 2 bytes
 - U+00EA → nhị phân **1110 1010**
 - UTF-8 encoding của 2 bytes: **110yyyyy 10xxxxxx**
 - Lấy tất cả 8 bits 1110 1010 của code point đưa vào các giá trị y x theo giá trị từ thấp đến cao
 - 6 bits của x: **101010**
 - 5 bits của y: **00011**
 - UTF-8 encoding: **11000011 10101010**
- Giá trị hex lưu xuống file của ký tự ê
 - **11000011 10101010 = C3AA**

Unicode: UTF-8 encoding

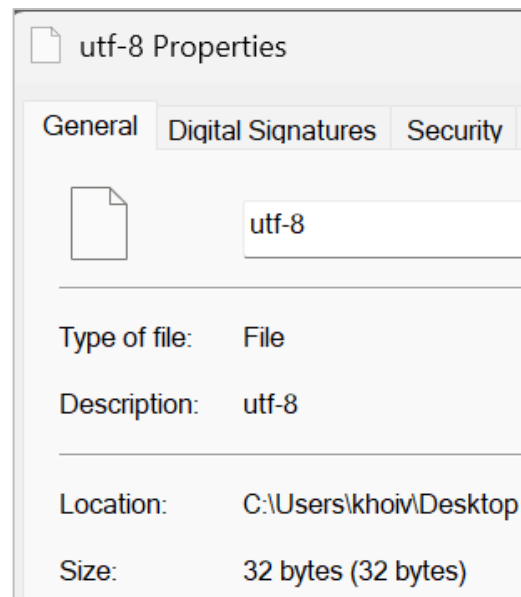
- Giá trị hex lưu xuống file của ký tự ê
 - 11000011 10101010 = C3AA



Unicode: UTF-8 encoding

- Ví dụ UTF-8 của chuỗi “Xử lý ngôn ngữ tự nhiên”

	X	ử		l	ý		n	g	ô	n		n	g	ữ		t	ự		n	h	i	ê	n
UTF-8	58	E1 BB AD	20	6C	C3 BD	20	6E	67	C3 B4	6E	20	6E	67	E1 BB AF	20	74	E1 BB B1	20	6E	68	69	C3 AA	6E

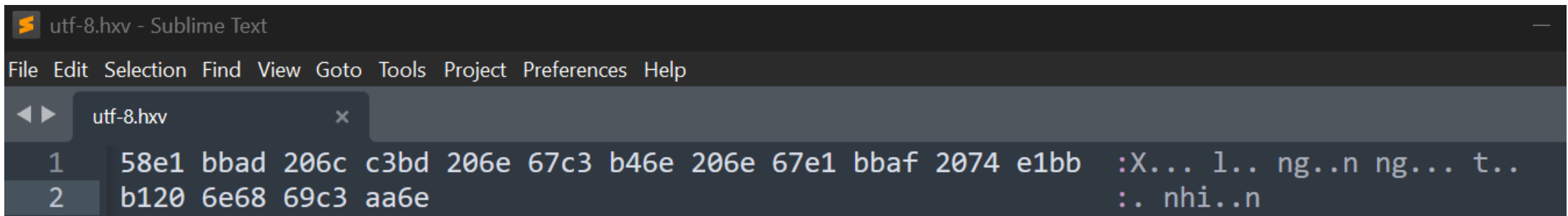


Unicode: UTF-8 encoding

■ Ví dụ UTF-8 của chuỗi “Xử lý ngôn ngữ tự nhiên”

58 E1 BB AD 20 6C C3 BD 20 6E 67 C3 B4 6E 20 6E
67 E1 BB AF 20 74 E1 BB B1 20 6E 68 69 C3 AA 6E

- Tổng cộng: 23 ký tự, 32 byte cho toàn bộ câu
- Các ký tự ASCII (X, l, n, g, t, h, i, khoảng trắng): 1 byte
 - $17 \times 1 = 17$ bytes
- Các ký tự có dấu (ý, ô, ê): 2 bytes
 - $3 \times 2 = 6$ bytes
- Các ký tự có dấu (ử, ữ, ự): 3 bytes
 - $3 \times 3 = 9$ bytes

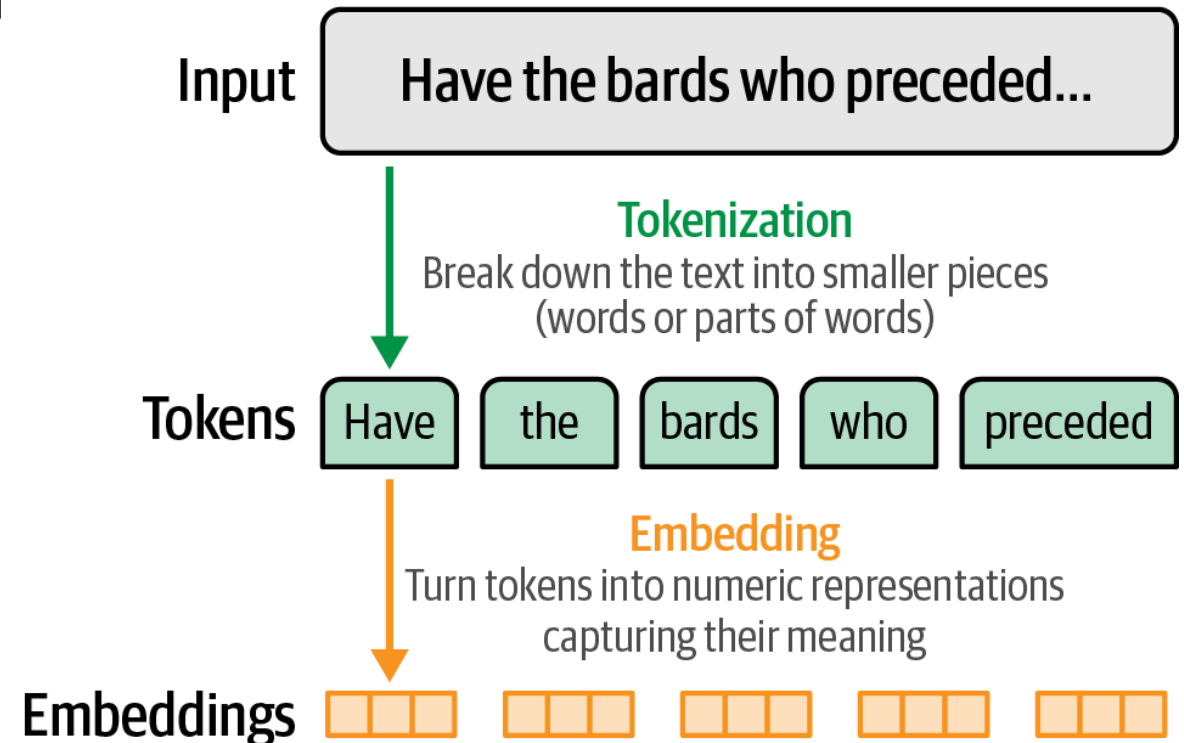
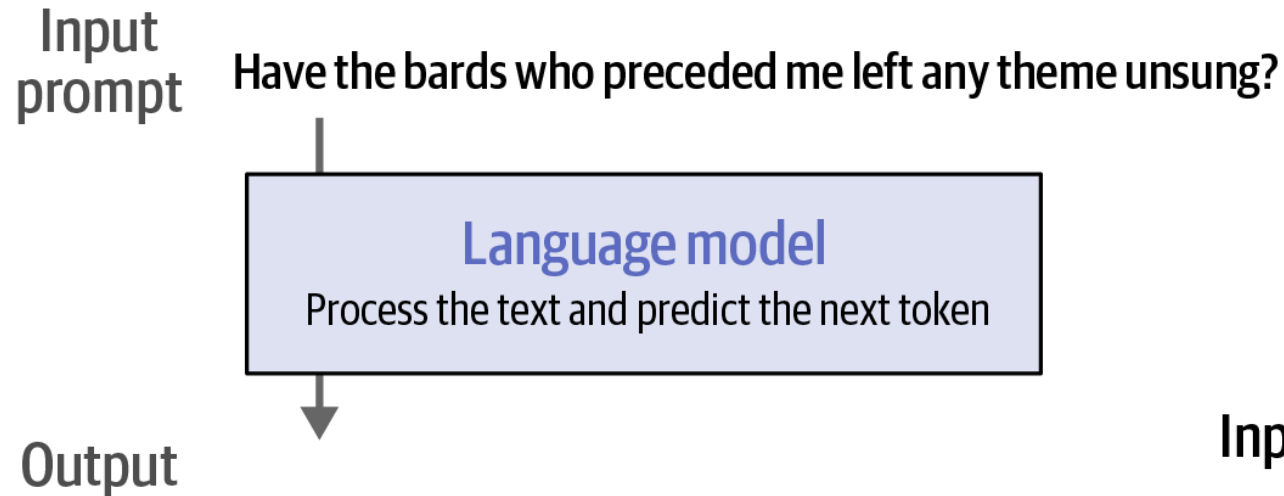


```
utf-8.hxv - Sublime Text
File Edit Selection Find View Goto Tools Project Preferences Help
utf-8.hxv x
1 58e1 bbad 206c c3bd 206e 67c3 b46e 206e 67e1 bbaf 2074 e1bb :X... l.. ng..n ng... t..
2 b120 6e68 69c3 aa6e :. nhi..n
```

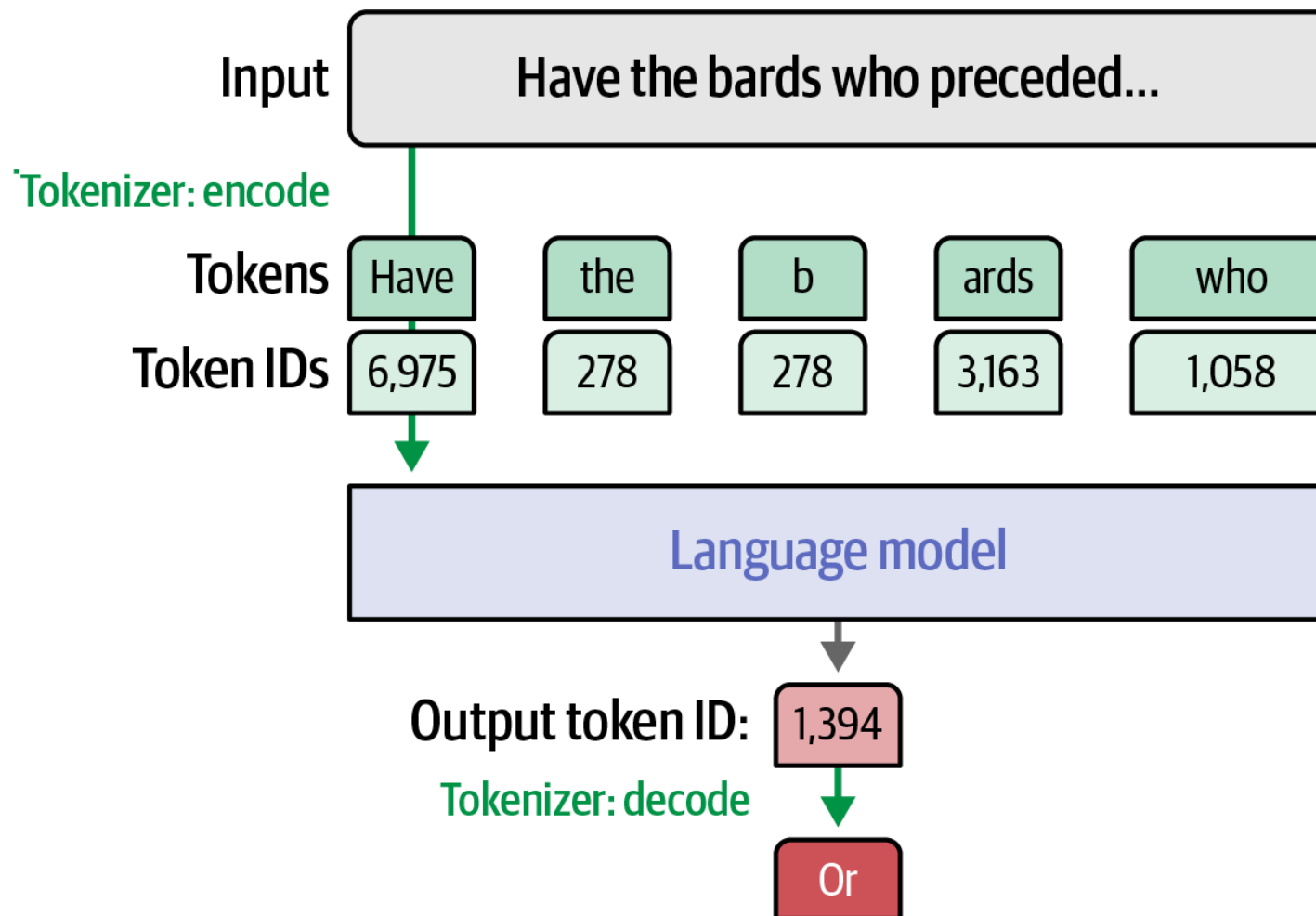
Unicode: python

- Chuỗi ký tự (strings) trong Python 3 được lưu dưới dạng Unicode
 - Mỗi chuỗi là một dãy (sequence) các code points
 - Các hàm xử lý chuỗi, biểu thức chính quy (**regex**), được áp dụng trực tiếp cho các code point
 - Hàm **len()** trả về độ dài của chuỗi theo code points, không phải bytes
 - **len("Xử lý ngôn ngữ tự nhiên") → 23**
- Các tập tin cần được mã hóa (encoded)/giải mã (decoded) khi đọc/ghi
 - Tất cả các tập tin được lưu trữ dưới một dạng mã hóa (encoding) nào đó
 - UTF-8, ASCII, ISO_8859_1,...

Tách từ (tokenization)



Tách từ (tokenization)



Tách từ (tokenization)

Text Have the 🎵 bards who preceded...

Word tokens



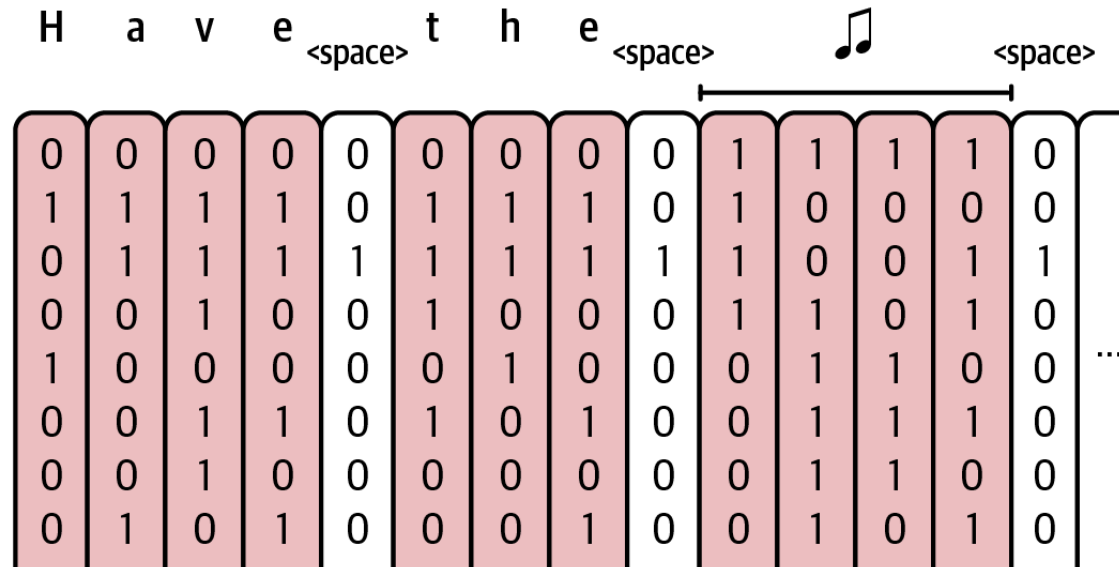
Subword tokens



Character tokens



Byte tokens



Tách từ (tokenization)

- Giai đoạn đầu tiên của Xử lý ngôn ngữ tự nhiên
- Quá trình phân đoạn/phân tách (**segmenting**) văn bản đầu vào (**input text**) thành các đơn vị (**unit**) nhỏ hơn gọi là **token**.
- Ví dụ, đoạn văn bản này dài bao nhiêu (có bao nhiêu đơn vị trong đó)?
 - “don’t” hay “New York” là 1 hay 2 token?
 - Sử dụng khoảng trắng (**white-space**) hay từ chính tả (**orthographic words**) [có trong tự điển]?
- Token có thể là:
 - Từ (**words**)
 - Hình vị (**morphemes**)
 - Ký tự (**characters**)

Tách từ (tokenization)

- Mỗi đơn vị có những khó khăn riêng
 - Từ và hình vị: khó định nghĩa (define) một cách chính thức
 - Ký tự: định nghĩa rõ ràng, nhưng quá nhỏ để làm token
- Vì vậy, sử dụng cách tiếp cận dựa trên dữ liệu (**data-driven approach**) để định nghĩa các token
 - Tạo các token có kích thước tương đương với từ hoặc hình vị, đôi khi sử dụng các đơn vị nhỏ hơn như ký tự.
- Đối với những từ chưa biết (**unknown word problem**) thì sao?
 - Những từ xuất hiện trong tập dữ liệu kiểm thử (test set) nhưng không có trong tập huấn luyện (training set).
- Cách tiếp cận dựa trên từ con (**subword**) là các token nhỏ hơn từ (words)
 - Subword có thể là chuỗi con (**substrings**) tùy ý, hoặc các đơn vị mang nghĩa (**meaning-bearing units**) như hình vị (**morphemes**) –est hoặc –er

Subword tokenization

- Hai thuật toán tách từ được sử dụng phổ biến trong các mô hình ngôn ngữ hiện đại (**modern language models**)
 - Byte-Pair Encoding (BPE) (Sennrich et al., 2016)
 - Unigram Language Modeling (ULM) (Kudo, 2018) (đôi khi bị nhầm lẫn gọi là "SentencePiece" theo tên thư viện chứa nó)
- Mỗi thuật toán đều có 2 phần:
 - Token trainer/learner
 - Nhận một tập dữ liệu huấn luyện thô và tạo ra một từ vựng (**vocabulary**), một tập hợp các token (**set of tokens**).
 - Token encoder/segmenter
 - Nhận một câu thử nghiệm thô và mã hóa nó theo từ vựng đó.

Byte Pair Encoding (BPE) token learner

- Lặp lại việc kết hợp (**merge**) các token liền kề thường xuyên (**frequent neighboring tokens**) để tạo ra các token dài hơn (**longer token**)
- Ví dụ cho ngữ liệu (**corpus**) ban đầu dài 10 ký tự: A B D C A B E C A B
 - Từ vựng gồm 5 ký tự: {A, B, C, D, E}
 - Cặp cái ký tự liền kề của ngữ liệu bao gồm: “A B”, “B D”, “D C”, “C A”, “A B”, “B E”, “E C”, “C A”, “A B”
 - cặp có tần suất nhiều nhất là “A B”: 3 lần → tạo được token mới là ‘AB’
 - từ vựng (vocabulary) mới (gồm 5 token) là: {A, B, C, D, E, AB}
 - Thay tất cả các cặp “A B” liền kề trong ngữ liệu với token mới ‘AB’
 - Corpus mới (dài 7 tokens): AB D C AB E C AB
- Lặp lại các bước trên ta được token mới ‘CAB’
 - Vocabulary (7 token): {A, B, C, D, E, AB, CAB}
 - Corpus mới (dài 5 tokens): AB D CAB E CAB

Byte-Pair Encoding (BPE) token learner

- Thuật toán tiếp tục đếm (**count**) và kết hợp (**merge**) để tạo ra các chuỗi ký tự/token dài hơn... Cho đến lần kết hợp thứ **k** (k là tham số của thuật toán)

Repeat:

- Choose most frequent neighboring pair ('A', 'B')
- Add a new merged symbol ('AB') to the vocabulary
- Replace every 'A' 'B' in the corpus with 'AB'.

Until *k* merges

Vocabulary

[A, B, C, D, E]

[A, B, C, D, E, **AB**]

[A, B, C, D, E, **AB**, **CAB**]

Corpus

A B D C A B E C A B

AB D C **AB** E C **AB**

AB D **CAB** E **CAB**

BPE token learner algorithm

- Phần huấn luyện của thuật toán BPE (Bostrom and Durrett (2020)):
 - dùng để xử lý một tập dữ liệu được chia nhỏ thành các ký tự hoặc byte riêng lẻ, và học từ vựng bằng cách hợp nhất các token một cách lặp đi lặp lại.
- V : từ vựng thu được bao gồm
 - Tập hợp các ký tự ban đầu cộng với k ký hiệu (symbols) mới

function BYTE-PAIR ENCODING(strings C , number of merges k) **returns** vocab V

```
 $V \leftarrow$  all unique characters in  $C$            # initial set of tokens is characters
for  $i = 1$  to  $k$  do                           # merge tokens  $k$  times
     $t_L, t_R \leftarrow$  Most frequent pair of adjacent tokens in  $C$ 
     $t_{NEW} \leftarrow t_L + t_R$                  # make new token by concatenating
     $V \leftarrow V + t_{NEW}$                      # update the vocabulary
    Replace each occurrence of  $t_L, t_R$  in  $C$  with  $t_{NEW}$  # and update the corpus
return  $V$ 
```

BPE token learner algorithm

- Điểm phức tạp duy nhất là trong thực tế, thay vì chạy trên chuỗi các ký tự thô, thuật toán chỉ chạy bên trong các từ (**inside words/within space-separated words**).
- Nghĩa là thuật toán **không** kết hợp (**merge**) các ký tự vượt qua ranh giới các từ (**word boundaries**)
 - Do đó, ngữ liệu đầu vào (**input corpus**) trước tiên thường được tách ra theo khoảng trắng (**white space**) và dấu câu (**punctuation**)
 - Sử dụng biểu thức chính quy (**regular expression**) để tách
 - Kết quả là tạo được tập hợp các chuỗi ban đầu (**starting set of strings**), mỗi chuỗi tương ứng với các ký tự của một từ (với khoảng trắng thường được gắn vào đầu từ), cùng với số lần xuất hiện (**tần suất**) của các từ.
 - Đếm số lần xuất hiện trong toàn bộ ngữ liệu (**corpus**), nhưng khi kết hợp (**merge**) chỉ thực hiện bên trong các chuỗi.

BPE token learner

- Cho ngữ liệu ban đầu: `set_new_new_renew_reset_renew`
- Đặt khoảng trắng (**space token**) vào đầu mỗi từ

corpus	vocabulary
2 <code>_ n e w</code>	<code>_, e, n, r, s, t, w</code>
2 <code>_ r e n e w</code>	
1 <code>s e t</code>	
1 <code>_ r e s e t</code>	

- Đếm tất cả các cặp ký tự kề (**all pairs of adjacent symbols**) → cặp `n e` xuất hiện nhiều nhất (**count** 4 = 2 `new` + 2 `renew`) → kết hợp/ghép `n e` thành `ne`

corpus	vocabulary
2 <code>_ ne w</code>	<code>_, e, n, r, s, t, w, ne</code>
2 <code>_ r e ne w</code>	
1 <code>s e t</code>	
1 <code>_ r e s e t</code>	

BPE token learner

- Lặp lại các bước trên cho cặp xuất hiện nhiều nhất **ne w** (count = 4) → **new**

corpus

2 _ new

2 _ r e new

1 s e t

1 _ r e s e t

vocabulary

_ , e , n , r , s , t , w , ne , new

- Lần ghép thứ 3: cặp **_ r** (count = 3) → **_r**
- Lần ghép thứ 4: cặp **_r e** (count = 3) → **_re**
 - Lúc này học được tiền tố (prefix) **re**

corpus

2 _ new

2 _re new

1 s e t

1 _re s e t

vocabulary

_ , e , n , r , s , t , w , ne , new , _r , _re

BPE token learner

■ Tiếp tục...

merge	current vocabulary
(<code>␣</code> , <code>new</code>)	<code>␣</code> , <code>e</code> , <code>n</code> , <code>r</code> , <code>s</code> , <code>t</code> , <code>w</code> , <code>ne</code> , <code>new</code> , <code>␣r</code> , <code>␣re</code> , <code>␣new</code>
(<code>␣re</code> , <code>new</code>)	<code>␣</code> , <code>e</code> , <code>n</code> , <code>r</code> , <code>s</code> , <code>t</code> , <code>w</code> , <code>ne</code> , <code>new</code> , <code>␣r</code> , <code>␣re</code> , <code>␣new</code> , <code>␣renew</code>
(<code>s</code> , <code>e</code>)	<code>␣</code> , <code>e</code> , <code>n</code> , <code>r</code> , <code>s</code> , <code>t</code> , <code>w</code> , <code>ne</code> , <code>new</code> , <code>␣r</code> , <code>␣re</code> , <code>␣new</code> , <code>␣renew</code> , <code>se</code>
(<code>se</code> , <code>t</code>)	<code>␣</code> , <code>e</code> , <code>n</code> , <code>r</code> , <code>s</code> , <code>t</code> , <code>w</code> , <code>ne</code> , <code>new</code> , <code>␣r</code> , <code>␣re</code> , <code>␣new</code> , <code>␣renew</code> , <code>se</code> , <code>set</code>

BPE encoder algorithm

- Sau giai đoạn học (BPE token learner/trainer) ta thu được danh sách các luật ghép (merge rules).
- Dùng các luật ghép để tách câu mới (tokenize a test sentence): chạy các luật ghép học được từ tập dữ liệu/ngữ liệu huấn luyện
 - Một cách tham lam (greedily), theo đúng thứ tự (order) chúng ta đã học
 - (tần suất các từ trong câu huấn luyện **không** đóng vai trò gì)

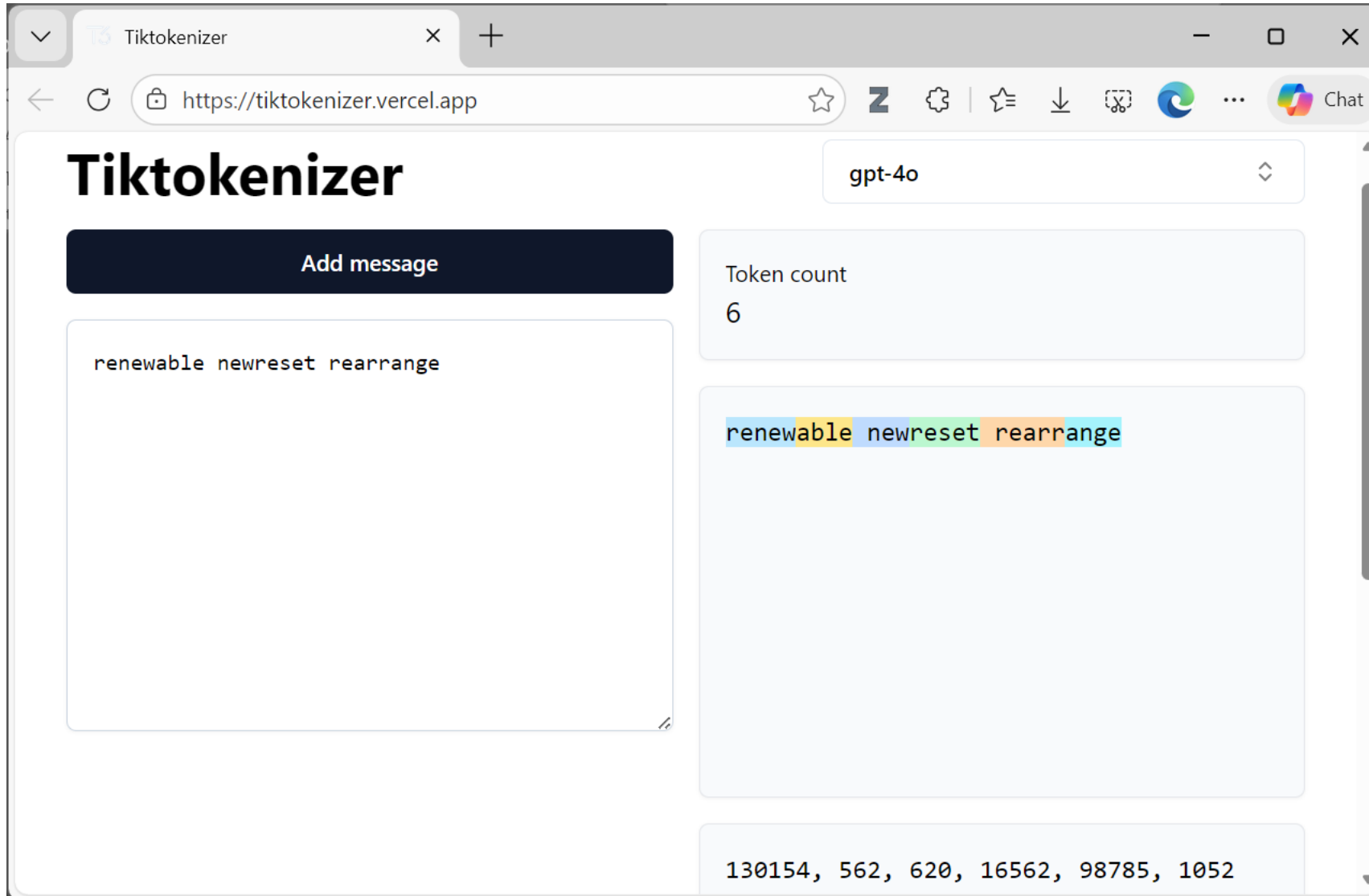
BPE encoder algorithm

merge	current vocabulary
(<code>▮</code> , <code>new</code>)	<code>▮</code> , <code>e</code> , <code>n</code> , <code>r</code> , <code>s</code> , <code>t</code> , <code>w</code> , <code>ne</code> , <code>new</code> , <code>▮r</code> , <code>▮re</code> , <code>▮new</code>
(<code>▮re</code> , <code>new</code>)	<code>▮</code> , <code>e</code> , <code>n</code> , <code>r</code> , <code>s</code> , <code>t</code> , <code>w</code> , <code>ne</code> , <code>new</code> , <code>▮r</code> , <code>▮re</code> , <code>▮new</code> , <code>▮renew</code>
(<code>s</code> , <code>e</code>)	<code>▮</code> , <code>e</code> , <code>n</code> , <code>r</code> , <code>s</code> , <code>t</code> , <code>w</code> , <code>ne</code> , <code>new</code> , <code>▮r</code> , <code>▮re</code> , <code>▮new</code> , <code>▮renew</code> , <code>se</code>
(<code>se</code> , <code>t</code>)	<code>▮</code> , <code>e</code> , <code>n</code> , <code>r</code> , <code>s</code> , <code>t</code> , <code>w</code> , <code>ne</code> , <code>new</code> , <code>▮r</code> , <code>▮re</code> , <code>▮new</code> , <code>▮renew</code> , <code>se</code> , <code>set</code>

- Ví dụ câu thử nghiệm: “renewable newreset rearrange”
- Trước tiên, tách mỗi từ [trong câu thử nghiệm] thành các ký tự/ký hiệu (characters)
- Áp dụng các luật ghép theo đúng thứ tự đã học: (1) `n e` → `ne`, (2) `ne w` → `new`, (3) `▮r`, (4) `▮re`, ...
- Kết quả
 - Tái tạo các từ trong tập huấn luyện (recreates training set words)
 - Nhưng cũng học được các từ con (subwords) như `▮re` có thể xuất hiện trong các từ mới (new words) như `rearrange`
 - thu được các token: `renew`, `able`, `new`, `reset`, `re`, `arrange`

BPE encoder algorithm

<https://tiktokenizer.vercel.app/>



The screenshot shows the Tiktokenizer web application interface. The browser address bar displays `https://tiktokenizer.vercel.app`. The application title is "Tiktokenizer". A dark blue button labeled "Add message" is visible. The input text area contains the string "renewable newreset rearrange". To the right, a dropdown menu is set to "gpt-4o". Below the input, a box shows the "Token count" as 6. The output area displays the tokens: "renewable newreset rearrange", where each word is highlighted in a different color (blue, yellow, green, orange). At the bottom, the resulting token IDs are listed: "130154, 562, 620, 16562, 98785, 1052".

BPE and Unicode

- Tất nhiên, trong thực tế, BPE được chạy với hàng chục nghìn lần hợp nhất trên một kho ngữ liệu đầu vào rất lớn:
 - Để tạo ra kích thước từ vựng lên tới 50.000, 100.000, hoặc thậm chí 200.000 token
 - Kết quả là hầu hết các từ có thể được biểu diễn dưới dạng token đơn, và chỉ những từ hiếm gặp (và từ không xác định) mới cần được biểu diễn bằng nhiều token.
- BPE chạy trên từng byte riêng lẻ (**individual bytes**) của văn bản được mã hóa UTF-8
 - Không phải trên các ký tự (characters) Unicode
 - BPE tìm lại (**rediscovers**) các chuỗi UTF-8 2-byte và các chuỗi 3-byte phổ biến
 - Chỉ có 256 giá trị có thể có của một byte, vì vậy không có token không xác định
 - (BPE có thể học một vài chuỗi UTF-8 bất hợp lệ vượt qua ranh giới ký tự, nhưng chúng có thể được lọc bỏ)

Thuật toán BPE và SuperBPE

- Có thể thay đổi quá trình **pretokenization** này để cho phép các BPE tokens trải rộng trên nhiều từ (**span multiple words**). Ví dụ, các thuật toán **SuperBPE** (Liu et al., 2025) và **BoundlessBPE** (Schmidt et al., 2025):
 - Trước tiên tạo ra các BPE subword tokens thông thường bằng cách thực hiện **pretokenization**.
 - Sau đó, chúng chạy giai đoạn BPE thứ hai cho phép hợp nhất (**merges**) các khoảng trắng (**spaces**) và dấu chấm câu (**punctuation**).
 - Kết quả là một tập hợp lớn các tokens có thể hiệu quả hơn.

BPE: By the way, I am a fan of the Milky Way.

SuperBPE: By the way, I am a fan of the Milky Way.

Visualizing GPT4o tokens

Anyhow, · she ' s · seen · Jane ' s · 224123 · flowers · anyhow !

- Tokens: 11865, 8923, 11, 31211, 6177, 23919, 885, 220, 19427, 7633, 18887, 147065, 0
- Hầu hết các từ là tokens, có khoảng trắng ở đầu (**initial space**)
- Các phụ tố (clitics) như ' s
 - Được tách ra (segmented off) từ Jane
 - Nhưng là một phần của các từ thông dụng khác (**part of frequent words**) như she ' s
- Các số được tách thành từng nhóm 3 số (**segmented into chunks of 3 digits**)
- Anyhow và · anyhow được tách khác nhau
- Một phần của điều này là do quá trình tiền xử lý (**preprocessing**)
 - Các biểu thức chính quy (**regular expressions**) để chia nhóm chữ số (**chunking digits**), loại bỏ các phụ tố (**stripping clitics**)

Tat Dat Duong's [Tiktokenizer](#) visualizer

Tokenizing across languages

- Mặc dù các bộ BPE tokenizer hỗ trợ nhiều ngôn ngữ (**multilingual**)
- Dữ liệu huấn luyện LLM vẫn chủ yếu là tiếng Anh.
 - Hầu hết các BPE token được sử dụng cho tiếng Anh, dẫn đến số lượng ít hơn cho các ngôn ngữ khác.
 - Các từ trong các ngôn ngữ khác thường bị tách ra (split up).

Tokenization is better in English

- Một câu hướng dẫn công thức nấu ăn (recipe) bằng hai ngôn ngữ
 - Tiếng Anh: 18 tokens; không có **word** nào bị tách thành nhiều **tokens** (no words are split into multiple tokens).

In · a · deep · bowl, · mix · the · orange · juice · with · the · sugar, · ginger, · and · nutmeg.

- Tiếng Tây Ban Nha: 33 token; 6/16 words bị tách thành hai phần.

En · un · recipiente · hondo, · mezclar · el · jugo · de · naranja · con · el · azúcar, · jengibre, · y · nuez · moscada.

Tat Dat Duong's [Tiktokenizer](#) visualizer on GPT4o

Tokenizers

- OpenAI Platform Tokenizer
 - <https://platform.openai.com/tokenizer>
- Tiktokenizer
 - <https://tiktokenizer.vercel.app/>

Ngữ liệu (corpora/corpus)

- Từ ngữ không tự nhiên mà có!
- Một văn bản được tạo ra bởi:
 - một hoặc nhiều tác giả (specific writers) cụ thể,
 - vào một thời điểm cụ thể (specific time),
 - với một thể loại cụ thể (specific variety),
 - bằng một ngôn ngữ cụ thể (specific language),
 - cho một chức năng cụ thể (specific function).
- Các kho ngữ liệu khác nhau tùy thuộc vào các khía cạnh như:
 - Ngôn ngữ (**Language**): Có 7097 ngôn ngữ trên thế giới --> kiểm tra thuật toán trên nhiều ngôn ngữ (hiệu quả khác nhau trên các ngôn ngữ khác).
 - Sự đa dạng (**Variety**): các biến thể tiếng Anh của người Mỹ gốc Phi, Ấn...
 - Thể loại (**Genre**): bản tin, tiểu thuyết, bài báo khoa học, Wikipedia...
 - Thông tin nhân khẩu học của tác giả (**Author Demographics**): tuổi, giới tính, dân tộc, tình trạng kinh tế xã hội của người viết

Chuyển đổi mã ngôn ngữ (Code switching)

- Người nói sử dụng nhiều ngôn ngữ trong cùng một câu nói.
 - Điều này rất phổ biến trên toàn thế giới, đặc biệt là trong ngôn ngữ nói và các thể loại liên quan như nhắn tin và mạng xã hội.

Bảng dữ liệu ngữ liệu (Corpus datasheets)

- Gebru et al (2020), Bender và Friedman (2018)
- Động cơ (**motivation**)
 - Tại sao ngữ liệu được thu thập?
 - Do ai thu thập?
 - Ai tài trợ?
- Tình huống (**situation**)
 - Văn bản được viết trong tình huống nào?
- Quá trình thu thập (**collection process**)
 - Lấy mẫu như thế nào? Có sự đồng ý không? Xử lý sơ bộ?
 - Quá trình chú thích, sự đa dạng, nhân khẩu học (demographics)...

Regular expression (biểu thức chính quy)

- Ngôn ngữ hình thức (**formal language**) để xác định các chuỗi văn bản
- Một phần của mọi tác vụ xử lý văn bản
 - Là bước tiền xử lý hoặc định dạng văn bản hữu ích, ví dụ như đối với quá trình tiền mã hóa BPE (pre-tokenization).
- Công cụ được sử dụng rộng rãi trong thực tiễn (**industry**) và học thuật (**academics**)
 - Phân tích dữ liệu văn bản (in analyzing text data)
 - Thu thập dữ liệu web (web scraping)
 - Khớp mẫu (pattern-matching)
 - Tiền xử lý tách từ trong các mô hình ngôn ngữ (preprocessing step for tokenization in language models)

Regular expression

- Tìm kiếm một mẫu (**pattern**) trong một chuỗi (**string**)
- Ví dụ, hàm **search()** trong Python quét qua chuỗi và trả về kết quả khớp đầu tiên (first match) của mẫu trong chuỗi
 - `re.search(pattern, string)`
- Biểu thức chính quy trên chuỗi ký tự thô (**raw string**) có dấu ngoặc kép (**double quotes**)
 - `r"regex"`
- Chuỗi ký tự thô xử lý các dấu gạch chéo ngược \ (**backslashes**) như là các ký tự thông thường (**literal characters**)
 - Nhiều mẫu biểu thức chính quy sử dụng dấu \

Regular expression: Why raw strings?

- Regex và Python đều sử dụng dấu \ cho các ký tự đặc biệt. Nếu không sử dụng cú pháp chuỗi thô (**raw string notation**), phải gõ thêm dấu \
 - “\d+” để tìm kiếm 1 hoặc nhiều chữ số (digits)
 - “\n” trong Python có nghĩa là ký tự "xuống dòng", không phải là dấu gạch chéo theo sau là chữ "n".
 - cần “\\n” cho hai ký tự
- Thay vào đó, hãy sử dụng cú pháp chuỗi thô của Python cho regex:
 - r“[tT]he”
 - r“\d+” khớp với một hoặc nhiều chữ số
 - thay vì “\\d+”
- Ví dụ, mẫu (**pattern**) khớp chuỗi con (**substring**) Buttercup trong bất kỳ chuỗi nào, như “I’m called little Buttercup”
 - R”buttercup”

Regular Expressions: Disjunctions

- Biểu thức chính quy: phép tuyển (**disjunctions**)
 - Các chữ cái (letters) bên trong dấu ngoặc vuông (square brackets) []

Pattern	Matches
<code>r"[mM]ary"</code>	Mary or mary
<code>r"[1234567890]"</code>	Any one digit

- Dãy/phạm vi sử dụng dấu gạch ngang (ranges using the dash [A-Z])

Pattern	Matches	
<code>r"[A-Z]"</code>	An upper case letter	<u>D</u> renched Blossoms
<code>r"[a-z]"</code>	A lower case letter	<u>m</u> y beans were impatient
<code>r"[0-9]"</code>	A single digit	Chapter <u>1</u> : Down the Rabbit Hole

Regular Expressions: Negation in Disjunction

- Biểu thức chính quy: phủ định trong phép tuyển (**negation in disjunction**)
 - Dấu mũ ^ là ký tự đầu tiên (đứng đầu) **nằm trong** danh sách [] sẽ phủ định danh sách đó.
 - Lưu ý: Dấu mũ chỉ có nghĩa phủ định khi nó đứng đầu trong dấu ngoặc vuông []
 - Các ký tự đặc biệt (., *, +, ?) mất đi ý nghĩa đặc biệt của chúng bên trong dấu ngoặc vuông []

Pattern	Matches	Examples
<code>r"[^A-Z]"</code>	Not upper case	O <u>y</u> fn pripetchik
<code>r"[^Ss]"</code>	Neither 'S' nor 's'	<u>I</u> have no exquisite reason"
<code>r"[^.]"</code>	Not a period	<u>O</u> ur resident Djinn
<code>r"[e^]"</code>	Either e or ^	Look up <u>^</u> now

Regular Expressions: Kleene star & Kleene plus

baa

baaa

baaaa ...

- Kleene star * (0 hoặc nhiều ký tự trước đó)
- Kleene plus + (1 hay nhiều ký tự trước đó)

r"baaa*" baa baaa baaaa ...

r"baa+"



Stephen C Kleene

Regular Expressions: Wildcard & Anchors ^ \$

- Dấu chấm (period) có nghĩa là “any character”
 - `r"."` khớp với bất kỳ chuỗi ký tự nào
 - `r"."` khớp với bất kỳ chuỗi nào gồm 0 (chuỗi rỗng) hoặc nhiều ký tự
- Anchors (neo) ^ \$
 - `^`: neo đứng đầu/bắt đầu chuỗi
 - `$`: neo sau cùng/kết thúc chuỗi
 - Lưu ý: dấu mẫu ^ nằm ngoài danh sách []

Pattern	Matches
<code>r"^[A-Z]"</code>	<u>P</u> alo Alto
<code>r"\.\$"</code>	The end <u>.</u>
<code>r".\$"</code>	The end <u>?</u> The end <u>!</u>

Regular Expressions: More Disjunction

- Groundhog là tên khác của woodchuck!
- Ký hiệu pipe | cho phép tuyển (**disjunction**)

Pattern	Matches
<code>r"groundhog woodchuck"</code>	Woodchuck
<code>r"yours mine"</code>	Yours
<code>r"a b c"</code>	= [abc]
<code>r"[gG]roundhog [Ww]oodchuck"</code>	Woodchuck



Regular Expressions: Convenient aliases

Pattern	Expansion	Matches	Examples
<code>r"\d"</code>	<code>[0-9]</code>	Any digit	Fahren ⁴ heit 51
<code>r"\D"</code>	<code>[^0-9]</code>	Any non-digit	^B lue Moon
<code>r"\w"</code>	<code>[a-zA-Z0-9_]</code>	Any alphanumeric or -	^D aiyu
<code>r"\W"</code>	<code>[^\w]</code>	Not alphanumeric or _	Look [!]
<code>r"\s"</code>	<code>[\r\t\n\f]</code>	Whitespace (space, tab)	Look [_] up
<code>r"\S"</code>	<code>[^\s]</code>	Not whitespace	^L ook up

Quá trình lập để viết biểu thức regex

- Tìm tất cả các xuất hiện (**instances**) của từ “the” trong văn bản
 - **r"the"**
 - Các ví dụ viết hoa bị thiếu như “The”, tHE”...
 - **r"[tT]he"**
 - Trả về không chính xác như các từ “other” hoặc “Theology”
 - **r"\b[tT]he\b"**
 - **\b**: word boundary (biên của từ)

False positives and false negatives

- Các sai sót dương tính giả (false positives) và âm tính giả (false negatives)
- Quá trình chúng ta vừa thực hiện dựa trên việc sửa chữa 2 loại lỗi:
 - Không khớp với những chuỗi đáng lẽ phải khớp (The)
 - Âm tính giả (False negatives)
 - Khớp với các chuỗi đáng lẽ không nên khớp (there, then, other)
 - Dương tính giả (False positives)
- Trong NLP chúng ta luôn đối mặt với các loại lỗi này.
- Việc giảm tỷ lệ lỗi cho một ứng dụng thường liên quan đến 2 nỗ lực trái ngược nhau:
 - Tăng độ phủ (coverage hay recall): giảm thiểu âm tính giả
 - Tăng độ chính xác (accuracy hay precision): giảm thiểu dương tính giả

Regular expressions

- Biểu thức chính quy đóng vai trò quan trọng đến bất ngờ
- Được sử dụng rộng rãi trong cả giới học thuật (**academics**) và công nghiệp (**industry**)
 - Là một phần của hầu hết các tác vụ xử lý văn bản, ngay cả đối với các quy trình mô hình ngôn ngữ thần kinh lớn (**neural language model pipelines**)
 - bao gồm định dạng và tiền xử lý văn bản (**text formatting and pre-processing**)
 - Rất hữu ích cho việc phân tích dữ liệu của bất kỳ dữ liệu văn bản nào

Regex Substitutions in Python

- Thay thế (**substitute**) các từ xuất hiện trong chuỗi/văn bản thành từ khác
- Ví dụ, thay từ “cherry” thành “apricot”
 - `re.sub(r"cherry", r"apricot", string)`
- Chuyển chữ thường sang chữ hoa (upper case)
 - `re.sub(r"janet", r"Janet", string)`

Regex substitutions: capture groups

- Chuyển định dạng ngày
 - US format dates (mm/dd/yyyy) to EU format dates (dd-mm-yyyy)
- Định dạng US (Pattern to match US format)
 - `r"\d{2}/\d{2}/\d{4}"` hay `r"(\d\d/\d\d/\d\d\d\d)"`
- Làm sao chỉ định câu lệnh thay thế hoán đổi vị trí ngày và tháng?
 - Sử dụng dấu ngoặc (**parentheses**) để nhóm (**grouping**) và lưu (**capture/store**) giá trị đã khớp (**match**) khi tìm kiếm (**search**)
 - Mỗi nhóm (**group**) giá trị cần tìm có số thứ tự (**number**)
 - Tham chiếu đến các nhóm qua số thứ tự
- `re.sub(r"(\d{2})/(\d{2})/(\d{4})", r"\2-\1-\3", string)`
 - mm: group 1; dd: group 2; yyyy: group 3 tương ứng với các số 1, 2, 3

Regex substitutions: capture groups

```
1 import re
2
3 text = "Today's date is 08/03/2026"
4 # Change US format dates (mm/dd/yyyy) to EU format dates (dd-mm-yyyy)
5 formatted_text = re.sub(r"(\d{2})/(\d{2})/(\d{4})", r"\2-\1-\3", text)
6 print("Reformatted date:", formatted_text)
```

✓ 0.0s

Reformatted date: Today's date is 03-08-2026

Regex substitutions: capture groups

- Giả sử chúng ta không muốn lưu giữ (capture) thông tin của nhóm thì sao?
- Thêm 2 ký tự **?:** sau dấu ngoặc mở (của nhóm
 - `r"(?:some|a few) (people|cats) like some \1/"`
 - Khi này nhóm (some|a few) được bỏ qua → nhóm (people|cats) sẽ trở thành nhóm đầu tiên (first group) với số tham chiếu là 1
 - Số khớp đúng (matches)
 - some cats like some cats
 - Không thoả (but not)
 - some cats like some some
- `r"(\d\d/\d\d/\d\d\d\d\s+){14}(\d\d/\d\d/\d\d\d\d)"`
 - `\s+`: lặp lại 1 hoặc nhiều khoảng trắng (space)
 - `{14}`: lặp lại kết quả khớp 14 lần
 - Pattern trên chỉ lưu (store/capture) ngày thứ 15 trong group 1

Lookahead assertions

▪ (`?= pattern`)

- đúng nếu mẫu khớp, nhưng có độ rộng bằng 0, không di chuyển con trỏ ký tự về phía trước (`doesn't advance character pointer`)

▪ (`?! pattern`)

- đúng nếu mẫu không khớp (true if a pattern does not match)

▪ Làm sao bắt (`capture`) được từ đầu tiên (`first word`) trong dòng (`line`), nhưng không được bắt đầu bằng ký tự T

- `r"^(?![tT])(\w+)\b"`
 - `\w+`: 1 hoặc nhiều chữ cái, số, hoặc `_`

Simple Application: ELIZA

- **Substitutions** và **capture** groups hữu ích cho việc triển khai các chatbot có ý nghĩa lịch sử quan trọng như ELIZA
- Các hệ thống NLP sơ khai (early NLP system) mô phỏng nhà trị liệu tâm lý theo trường phái Rogerian (Rogerian psychotherapist)
 - Joseph Weizenbaum, 1966.
- Sử dụng phương pháp so khớp mẫu để đối sánh (**pattern matching**). Ví dụ:
“I need X” dịch thành câu “What would it mean to you if you got X?”

User₂: They're always bugging us about something or other.

ELIZA₂: CAN YOU THINK OF A SPECIFIC EXAMPLE

User₃: Well, my boyfriend made me come here.

ELIZA₃: YOUR BOYFRIEND MADE YOU COME HERE

User₄: He says I'm depressed much of the time.

ELIZA₄: I AM SORRY TO HEAR YOU ARE DEPRESSED

```
re.sub(r".* YOU ARE (DEPRESSED|SAD) .*",r"I AM SORRY TO HEAR YOU ARE \1",input)
re.sub(r".* YOU ARE (DEPRESSED|SAD) .*",r"WHY DO YOU THINK YOU ARE \1",input)
re.sub(r".* ALWAYS .*",r"CAN YOU THINK OF A SPECIFIC EXAMPLE",input)
```

Pretokenization for BPE

- Trong thuật toán BPE (đề cập ở phần trước), pre-tokenization:
 - Ngữ liệu (**corpus**) ban đầu được tách thô bằng khoảng trắng → Sau đó thuật toán BPE tạo các tokens từ chuỗi các ký tự trong các từ.
 - Lưu ý, không tokenize vượt qua giới hạn của từ.
- Mô hình ngôn ngữ GPT-2 sử dụng **Regular expression** để pre-tokenization.
- Các quy tắc GPT-2 để tiền mã hóa (**pretokenization**):
 - Đếm các từ phụ (**clitics**) như 't và 'm như các tokens
 - Tách dấu câu (**punctuation**) và số (**numbers**)
- Thực hiện 2 bước trên với một biểu thức regex sau:
`r"'s|'t|'re|'ve|'m|'ll|'d| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+|\s+(?!\\S)|\s+"`

Pretokenization for BPE

- Trong thư viện chuẩn **re** của Python, các escape sau KHÔNG được hỗ trợ (sẽ báo lỗi bad escape \p)
 - \p{L} (Unicode Letter)
 - \p{N} (Unicode Number)
- Do đó, cài đặt thư viện **regex** để hỗ trợ đầy đủ **Unicode**

```
1 import regex as re
2
3 pattern = r"'s|'t|'re|'ve|'m|'ll|'d| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+|\s+(?!\S)|\s+"
4 text = "Chào các bạn lớp Xử lý ngôn ngữ tự nhiên AI24111, ngành TTNT khóa 2024-2029."
5 tokens = re.findall(pattern, text)
6 print("Tokens:", tokens)
```

- Trả về (output) các tokens (20):

'Chào', ' các', ' bạn', ' lớp', ' Xử', ' lý', ' ngôn', ' ngữ', ' tự', ' nhiên',
' AI', '24111', ', ', ' ngành', ' TTNT', ' khóa', ' 2024', '-', '2029', '.'

Thư viện Python regex (thay vì thư viện re)

■ Các toán tử đặc biệt (operators) `\p` và `\P`

- `\p{L}` khớp với bất kỳ chữ cái Unicode nào (any Unicode letter)
- `\P{L}` khớp với bất kỳ ký tự nào không phải chữ cái (any non-letter)
- `\p{N}` khớp với bất kỳ số nào (any number)
- `\P{N}` khớp với bất kỳ ký tự nào không phải số (any non-number)

■ Trong trường hợp nếu phải dùng thư viện `re` thì thay:

- `\p{L}` thành `[^\W\d_]`
- `\p{N}` thành `\d`

The GPT-2 pre-tokenizer regular expression, used to split (roughly) on whitespace before running the BPE algorithm

```
1 import regex as re
2
3 pat = re.compile(
4     # Contractions: 't and 'm are tokens
5     r"'s|'t|'re|'ve|'m|'ll|'d|"
6     # Words: sequence of Unicode letters (after optional space)
7     r" ?\p{L}+|"
8     # Number: sequence of digits (after optional space)
9     r" ?\p{N}+|"
10    # Punctuation: sequence of non-alphanumeric/non-space
11    # (after optional space)
12    r" ?[^\s\p{L}\p{N}]+|"
13    # whitespace
14    r"\s+(?!\\S)|\s+"
15 )
16 text = "Chào các bạn lớp Xử lý ngôn ngữ tự nhiên AI24111, ngành TTNT khóa 2024-2029."
17 print("Number of tokens:", len(tokens))
18 print(pat.findall(text))
```

BPE & SuperBPE

- Thuật toán **SuperBPE** (*Liu et al., 2025*) và **BoundlessBPE** (*Schmidt et al., 2025*)
 - Trước tiên, tạo ra các token từ con BPE thông thường (regular BPE subword tokens) bằng cách thực hiện quá trình pre-tokenization.
 - Giai đoạn 2 của BPE cho phép hợp nhất (merge) các khoảng trắng (spaces) và dấu chấm câu (punctuation). Kết quả là tập hợp lớn các token được sử dụng hiệu quả hơn.

BPE: By the way, I am a fan of the Milky Way.

SuperBPE: By the way, I am a fan of the Milky Way.

Rule-based tokenization (tách từ dựa trên luật)

- Mặc dù việc phân tách từ thành các từ nhỏ (**subword tokenization**) là tiêu chuẩn (**norm**)
- Tuy nhiên, đôi khi chúng ta cần các từ cụ thể (**particular tokens**)
- Ví dụ như trong phân tích cú pháp (**parsing**), nơi trình phân tích (**parser**) cần các từ ngữ pháp (**grammatical words**), hoặc trong khoa học xã hội (**social science**)

Issues for rule-based tokenization

- Hầu như không phải lúc nào cũng loại bỏ các dấu chấm câu (**punctuation**):
 - m.p.h., Ph.D., AT&T, cap'n
 - prices (\$45.55)
 - dates (01/02/06)
 - URLs (http://www.stanford.edu)
 - hashtags (#nlproc)
 - email addresses (someone@cs.colorado.edu)
- Các số được tách khác nhau trong các ngôn ngữ
 - English 555,500.50 = French 555 500,50
- Biểu thức nhiều từ (Multiword expressions – MWE)?
 - New York, rock 'n' roll
 - Tùy thuộc vào ứng dụng, các thuật toán tokenization có thể tách các multiword expressions trên thành một token đơn (single).

Do đó, việc mã hóa dựa trên quy tắc có mối liên hệ mật thiết với việc nhận dạng thực thể được đặt tên.

Penn Treebank Tokenization Standard

- Một tiêu chuẩn tokenization thường được sử dụng là tiêu chuẩn Penn Treebank tokenization
 - Được dùng cho các kho ngữ liệu đã được phân tích cú pháp (treebank) do Linguistic Data Consortium (LDC) phát triển, nguồn gốc của nhiều bộ dữ liệu hữu ích.
 - Ví dụ, tách các clitics như `doesn't` → `does` + `n't`

Input: "The San Francisco-based restaurant," they said,
"doesn't charge \$10".

Output: "_The_San_Francisco-based_restaurant_,_"_they_said_,_
"_does_n't_charge_\$_10_"_.

Tokenization in NLTK

- *Bird, Loper and Klein (2009), Natural Language Processing with Python. O'Reilly*

```
1 # NLTK (The Natural Language Toolkit)
2 # is a Python package for natural language processing)
3 import nltk
4
5 text = "That U.S.A. poster-print costs $12.40..."
6 pattern = r"""(?x) # set flag to allow verbose regexps
7     (?:[A-Z]\.)+ # abbreviations, e.g. U.S.A.
8     | \w+(?:-\w+)* # words with optional internal hyphens
9     | \$?\d+(?:\.\d+)?%? # currency, percentages, e.g. $12.40, 82%
10    | \.\.\. # ellipsis
11    | [][.,;'"?():_`-] # these are separate tokens; includes ], [
12    """
13 nltk.regexp_tokenize(text, pattern)
```

✓ 58.7s

```
['That', 'U.S.A.', 'poster-print', 'costs', '$12.40', '...']
```

Sentence Segmentation (Tách câu)

- !, ? thường không gây nhầm lẫn (**unambiguous**) nhưng dấu chấm (**period**)
“.” rất mơ hồ (**very ambiguous**)
 - Ranh giới câu (**Sentence boundary**)
 - Các từ viết tắt (**Abbreviations**) như Inc. hoặc Dr.
 - Các con số như .02% hoặc 4.3
- Thuật toán phổ biến (**Common algorithm**):
 - Phân tách từ trước (**Tokenize first**): Sử dụng các quy tắc (**rules**) hoặc học máy (**machine learning**) để phân loại dấu chấm là:
 - (a) một phần của từ hoặc
 - (b) ranh giới câu
 - Có thể dùng một từ điển viết tắt (**abbreviation dictionary**)
- Việc tách câu sau đó thường có thể được thực hiện bằng các quy tắc dựa trên quá trình tách từ này.

Space-based tokenization

- Tách từ dựa vào khoảng trắng
- Một cách rất đơn giản để phân tách từ
 - Đối với các ngôn ngữ sử dụng khoảng trắng (spaces) giữa các từ (words)
 - Hệ thống chữ viết (writing systems) dựa trên tiếng Ả Rập, Cyrillic, Hy Lạp, Latinh, v.v.
 - Tách một từ giữa các vị trí có khoảng trắng
- Công cụ Unix để phân tách từ dựa trên khoảng trắng
 - Lệnh "tr"
 - Lấy cảm hứng từ cuốn **UNIX for Poets** của **Ken Church**
 - Cho một tập tin văn bản, xuất ra (output) các từ và tần suất xuất hiện (frequencies) của chúng.

Simple Tokenization in UNIX

```
# tr -sc 'A-Za-z' '\n' < shakes.txt  
| sort  
| uniq -c
```

Change all non-alpha to newlines

Sort in alphabetical order

Merge and count each type

1945 A	25 Aaron
72 AARON	6 Abate
19 ABBESS	1 Abates
5 ABBOT	5 Abbess
...	6 Abbey
	3 Abbot
	...

The first step: tokenizing

```
# tr -sc 'A-Za-z' '\n' < shakes.txt | head
```

THE

SONNETS

by

William

Shakespeare

From

fairest

creatures

We

...

The second step: sorting

```
# tr -sc 'A-Za-z' '\n' < shakes.txt | sort | head
```

A

A

A

A

A

A

A

A

A

...

More counting

- Kết hợp chữ hoa và chữ thường (Merging upper and lower case)

```
tr 'A-Z' 'a-z' < shakes.txt | tr -sc 'A-Za-z' '\n' | sort | uniq -c
```

- Sắp xếp theo tần suất các từ (Sorting the counts)

```
tr 'A-Z' 'a-z' < shakes.txt | tr -sc 'A-Za-z' '\n' | sort | uniq -c |  
sort -n -r
```

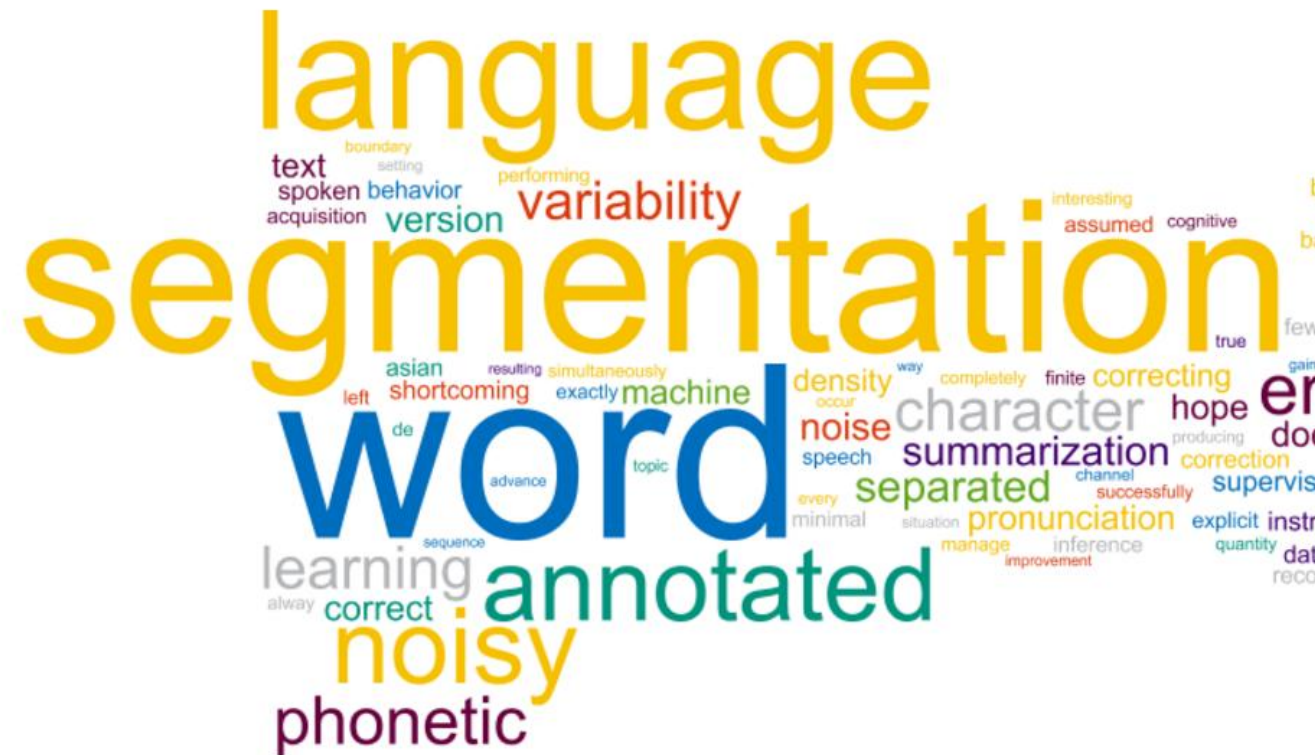
```
23243 the  
22225 i  
18618 and  
16339 to  
15687 of  
12780 a  
12163 you  
10839 my  
10005 in  
8954 d
```

What happened here?

Tách từ tiếng Việt (Vietnamese word segmentation)

Nội dung

- Quy tắc cấu tạo từ tiếng Việt
- Các hướng tiếp cận tách từ trong văn bản tiếng Việt
- Một số công cụ tách từ tiếng Việt



- Cách tách từ cho Tiếng Việt
 - <https://streetcodevn.com/blog/vntok>
- Vietnamese Word tokenization
 - <https://www.kaggle.com/code/boyle01/vietnamese-word-tokenization>
- NlpHUST/vi-word-segmentation
 - <https://huggingface.co/NlpHUST/vi-word-segmentation>
- VnCoreNLP: A Vietnamese natural language processing toolkit
 - <https://github.com/vncorenlp/VnCoreNLP?tab=readme-ov-file#python>

Tách từ tiếng Việt

- Mục tiêu của việc tách từ văn bản đầu vào là để khử tính nhập nhằng về ngữ nghĩa của văn bản.
- Xoài phun thuốc sâu không ăn.
 - Xoài / phun thuốc / sâu / không / ăn.
 - Xoài / phun / thuốc sâu / không / ăn.
- Ăn cơm không được uống rượu.
 - Ăn / cơm / không / được / uống / rượu.
 - Ăn / cơm không / được / uống / rượu.
- Mẹ vào ca ba con ngủ với dì.
 - Mẹ / vào / ca ba / con / ngủ / với / dì.
 - Mẹ / vào / ca / ba con / ngủ / với / dì.

Quy tắc cấu tạo từ tiếng Việt

- Tiếng Việt là ngôn ngữ không biến hình
 - Từ không biến đổi hình thái
- Đơn vị cơ bản:
 - tiếng/âm tiết (syllable) $\neq$ từ (word)
- Từ có thể gồm:
 - một tiếng: ăn, học
 - nhiều tiếng: sinh viên, công nghệ thông tin
- Vấn đề
 - Không có dấu hiệu hình thái rõ ràng để phân tách từ
- Khoảng 85% các loại từ tiếng Việt được cấu thành từ ít nhất hai âm tiết
- “thuế thu nhập cá nhân” (individual cá_nhân income thu_nhập tax thuế)
 - Gồm 5 âm tiết (syllables) tạo thành một cụm từ gồm 2 từ (a two-word phrase) “thuế_thu_nhập” và “cá_nhân”

Quy tắc cấu tạo từ tiếng Việt

■ Từ đơn

- Một tiếng/âm tiết: đi, học, đẹp

■ Từ ghép

- Ghép nhiều tiếng có quan hệ nghĩa
- Ghép chính phụ: nhà cửa, học sinh
- Ghép đẳng lập: bàn ghế, ăn uống

■ Từ láy

- Lặp lại âm để tạo sắc thái
- Láy toàn bộ: xanh xanh
- Láy bộ phận: lấp lánh, lung linh

■ Từ mượn

- Hán Việt: giáo dục, công nghiệp
- Âu hóa: internet, video

■ Hệ quả trong NLP

- Khoảng trắng không phải ranh giới từ
- Ví dụ: “học sinh giỏi”
- Có thể là:
 - [học_sinh] [giỏi]
 - [học] [sinh] [giỏi] (sai)

■ Vì vậy cần word segmentation

Các hướng tiếp cận tách từ tiếng Việt

■ Rule-based (dựa trên luật)

- Dựa vào từ điển
- Quy tắc ngữ pháp
- Maximum Matching (Forward/Backward)

■ Ưu điểm

- Dễ hiểu
- Không cần dữ liệu huấn luyện

■ Nhược điểm

- Không xử lý tốt nhập nhằng
 - “sinh viên học sinh”
- Phụ thuộc từ điển

Các hướng tiếp cận tách từ tiếng Việt

■ Statistical / Machine Learning

- Mô hình chuỗi (Sequence labeling): biến bài toán thành gán nhãn

■ Các mô hình

- HMM (Hidden Markov Model)
- CRF (Conditional Random Fields)

■ Ưu điểm

- Xử lý tốt nhập nhằng
- Có khả năng học từ dữ liệu

Các hướng tiếp cận tách từ tiếng Việt

■ Deep Learning (hiện đại)

a. BiLSTM + CRF

- BiLSTM: học ngữ cảnh hai chiều
- CRF: đảm bảo cấu trúc nhãn hợp lệ
- Đây là baseline mạnh trong nhiều năm

b. Transformer-based (state-of-the-art)

- Các mô hình:
 - BERT-based
 - PhoBERT (pretrained cho tiếng Việt)
- Cách tiếp cận
 - Fine-tune như bài toán sequence labeling
- Ưu điểm
 - Hiểu ngữ cảnh sâu
 - Xử lý tốt nhập nhằng (**ambiguity**)

Các hướng tiếp cận tách từ tiếng Việt

▪ Subword Tokenization (hiện đại)

a. Byte Pair Encoding (BPE)

- Tách theo thống kê
- Dùng trong:
 - GPT
 - Transformer

b. SentencePiece / Unigram LM

- Khác với word segmentation:
 - Không cố gắng tạo “từ đúng”
 - Tối ưu cho mô hình học

Một số công cụ tách từ tiếng Việt

■ Công cụ truyền thống

1. VnTokenizer

- Dựa trên CRF
- Khá cũ nhưng ổn định

2. JVNsegmenter

- Một trong những công cụ đầu tiên
- Dựa trên Học máy (ML) cổ điển

Một số công cụ tách từ tiếng Việt

■ Công cụ hiện đại

1. PyVi

2. VnCoreNLP

- Bộ NLP mạnh của Việt Nam
- Bao gồm:
 - Word segmentation
 - POS tagging
 - NER, dependency parsing
- Ưu điểm:
 - Nhanh
 - Chính xác cao

■ RDRSegmenter

- Module tách từ trong VnCoreNLP
- Kết hợp rule-based + supervised learning
- Được dùng rộng rãi trong các hệ thống hiện đại

3. Underthesea

Một số công cụ tách từ tiếng Việt

■ Công cụ dựa trên Deep Learning

1. PhoBERT (VinAI)

- Không phải tokenizer truyền thống
- Dùng:
 - SentencePiece
 - Subword

■ Pretrained Vietnamese models

- PhoBERT
- ViT5
- ViGPT

■ Xu hướng:

- End-to-end learning
- Không cần pipeline tách từ riêng

A Fast and Accurate Vietnamese Word Segmenter

Dat Quoc Nguyen¹, Dai Quoc Nguyen², Thanh Vu³, Mark Dras⁴, Mark Johnson⁴

¹The University of Melbourne, Australia; ²Deakin University, Australia;

³Newcastle University, United Kingdom; ⁴Macquarie University, Australia

`dqnguyen@unimelb.edu.au, dai.nguyen@deakin.edu.au,`
`thanh.vu@newcastle.ac.uk, {mark.dras, mark.johnson}@mq.edu.au`

Abstract

We propose a novel approach to Vietnamese word segmentation. Our approach is based on the Single Classification Ripple Down Rules methodology (Compton and Jansen, 1990), where rules are stored in an exception structure and new rules are only added to correct segmentation errors given by existing rules. Experimental results on the benchmark Vietnamese treebank show that our approach outperforms previous state-of-the-art approaches JVNsegmenter, vnTokenizer, DongDu and UETsegmenter in terms of both accuracy and performance speed. Our code is open-source and available at: <https://github.com/datquocnguyen/RDRsegmenter>.

Keywords: Vietnamese, Word segmentation, Single classification ripple down rules

